

LP SERIES • AI/ML & SYSTEMS LEARNING NOTES

Types of APIs

REST · GraphQL · WebSocket · Webhooks · gRPC · SOAP — fact-checked, expanded, and interview-ready notes with real-world examples.

REST

GraphQL

WebSocket

Webhooks

gRPC

SOAP

Contents

01	What Is an API?	Foundations
02	REST API	Methods, JWT, Pagination
03	GraphQL	Query · Mutation · Subscription
04	WebSocket	Real-time, Full-Duplex
05	Webhooks	Event-driven, Server-to-Client
06	gRPC	Protobuf, HTTP/2, Microservices
07	SOAP	XML, WS-Security, Enterprise/Banking
08	API Gateway & Infrastructure	Rate Limiting, Auth, Monitoring
09	API Frameworks	Flask, FastAPI, Express & More
10	Expert Insights & Comparison	Deeper Context & Cross-Concepts
11	Interview Questions	8 Q&As
12	Quick Revision & Key Takeaways	Summary Cards

What Is an API?

One-line definition: An API (Application Programming Interface) is a contract/middleman that lets a client (frontend/app) request data or actions from a server (backend), without the client needing to know the server's internal implementation.

ANALOGY

You (client) → Waiter (API) → Chef (server/database). You never talk directly to the chef — you tell the waiter what you want, and the waiter relays it, then brings back your food (response).

Why It Exists

Without an API, every frontend would need direct, tightly-coupled access to backend databases/services — a security and maintenance nightmare. APIs decouple frontend from backend, enforce security boundaries, and let multiple clients (mobile app, web app, third parties) reuse the same backend logic.

Not all APIs are network APIs — a library function call is technically an API too — but this document focuses specifically on the **network/web APIs** used for client–server communication over the internet.

API Styles at a Glance

Type	Data Format	Communication Style	Best For
REST	JSON (mostly)	Request → Response	Public web/mobile APIs
GraphQL	JSON	Client-defined query → Response	Complex UIs, mobile bandwidth-saving
WebSocket	Text/Binary frames	Persistent, two-way	Chat, live tracking, gaming
Webhooks	JSON (mostly)	Server-initiated push	Payment/event notifications
gRPC	Protobuf (binary)	RPC-style, streaming	Internal microservices
SOAP	XML	Request → Response	Banking, legacy ERP

REST API

Full form: REpresentational State Transfer | **Origin:** Roy Fielding's 2000 doctoral dissertation.

Core Components

Component	Purpose
URL (endpoint)	Identifies where the resource lives on the server
HTTP Method	Identifies what action to perform
Headers	Metadata — content type, authorization, caching
Body	Actual payload sent (for POST / PUT / PATCH)

URL Anatomy

```
https://api.zomato.com/v1/restaurants/1234/reviews?sort=recent
```

Segment	Value	Meaning
Protocol	<code>https://</code>	Secure HTTP
Server address	<code>api.zomato.com</code>	Where the API lives
Version	<code>/v1</code>	API version — lets the API evolve without breaking older clients
Resource	<code>/restaurants</code>	The type of thing being accessed
Resource ID	<code>/1234</code>	A specific restaurant
Sub-resource	<code>/reviews</code>	Something belonging to that restaurant
Query string	<code>?sort=recent</code>	Extra instructions (filtering, sorting, pagination)

Query Parameters & Pagination

Query parameters refine a GET request without changing the endpoint path:

```
GET /v1/posts?filter=trending&sort=recent&page=2&page_size=20
```

Parameter	Purpose
<code>filter</code>	Narrows results to a subset matching a condition
<code>sort</code>	Orders the results (e.g., newest first)

`page /` Pagination — the server sends one "page" of results at a time instead of the entire dataset, protecting both server load and client bandwidth
`page_size`

HTTP Methods (CRUD Mapping)

Method	Purpose	Idempotent?
GET	Retrieve data	Yes
POST	Create new data	No
PUT	Full replace of existing data	Yes
PATCH	Partial update to existing data	No
DELETE	Remove data	Yes

PATCH updates part of a resource, while **PUT** replaces the entire resource — this distinction is a very common interview trap.

Headers — Common Fields

- `Content-Type` — format of data being sent, e.g. `application/json`
- `Authorization` — identity proof, typically `Bearer <token>`
- `Accept` — response format the client wants back

JWT — Why "Bearer Token" Auth Is So Common

A **JWT (JSON Web Token)** is a compact, encoded string with three parts — header, payload, signature — that together carry: **who you are**, **what you're allowed to do**, and an **expiry time**. Because the token is cryptographically signed by the server, the server can verify it's genuine just by checking the signature — **it doesn't need to look the user up in a database on every request**. This is what makes JWT-based auth stateless, fast, and horizontally scalable.

Stateless also means a JWT can't easily be "revoked" before it expires — if a token is stolen, it stays valid until its expiry unless you maintain a server-side blocklist (which reintroduces some statefulness). This is why short expiry times plus a refresh-token flow are standard practice.

Request–Response Flow (Zomato Example)

- 1 Client sends `GET /restaurants?search=biryani` with an `Authorization` header
- 2 Server validates the token (AuthN — who are you)
- 3 Server checks permissions (AuthZ — are you allowed)
- 4 Server queries the database
- 5 Server responds with a JSON payload + HTTP status code

REST typically runs over HTTP/1.1, which supports persistent connections but not true **multiplexing** — sending many requests over one connection simultaneously is an **HTTP/2** feature, used later by gRPC.

Worked Example — Full Request & Response

A mobile app fetching a user's order history:

```
GET /v1/users/482/orders?status=delivered HTTP/1.1
Host: api.zomato.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{
  "orders": [
    { "id": "ORD1092", "restaurant": "Biryani House", "total": 349, "status": "delivered" },
    { "id": "ORD1085", "restaurant": "Pizza Corner", "total": 599, "status": "delivered" }
  ],
  "page": 1,
  "has_more": false
}
```

Common HTTP Status Codes

Code	Meaning	Typical Cause
200 OK	Success	Request completed normally
201 Created	Resource created	Successful POST
400 Bad Request	Client error	Malformed request/invalid input
401 Unauthorized	Not authenticated	Missing/invalid token
403 Forbidden	Not authorized	Valid identity, insufficient permission
404 Not Found	Resource doesn't exist	Wrong URL/ID
429 Too Many Requests	Rate limited	Client exceeded quota
500 Internal Server Error	Server-side failure	Unhandled exception/bug
504 Gateway Timeout	Upstream didn't respond in time	Slow downstream service

Advantages

- Simple, human-readable, widely supported
- Stateless — scales horizontally with ease
- Huge tooling ecosystem (Postman, Swagger)

Limitations

- Over-fetching — returns more data than needed
- Under-fetching — needs multiple round trips
- Versioning can get messy

GraphQL

Origin: Developed internally at Facebook starting 2012 to solve mobile News Feed performance problems; publicly released and open-sourced in 2015.

Problem It Solves

REST returning a fixed data shape forces either **over-fetching** (getting name, photo, age, ID, branch when you only wanted name + photo) or **building multiple REST endpoints** for different field combinations. GraphQL solves this with a **single endpoint** where the client specifies exactly which fields it needs.

Core Components

Component	Analogous To	Purpose
Query	GET	Fetch data
Mutation	POST / PUT / DELETE	Change data
Subscription	WebSocket	Real-time updates on data change

Example

```
query {  
  employee(id: "123") {  
    name  
    photo  
  }  
}
```

The response contains **exactly** `name` and `photo` — nothing more:

```
{  
  "data": {  
    "employee": { "name": "Anita Rao", "photo": "https://cdn.../anita.jpg" }  
  }  
}
```

REST vs GraphQL — Same Task, Different Shape

	REST	GraphQL
Endpoints needed	3 separate (name+photo, +salary, +branch)	1 endpoint, flexible query
Response size	Fixed — all fields every time	Only requested fields
Client control	Server decides shape	Client decides shape

Advantages

- No over/under-fetching
- Strongly typed, self-documenting schema
- One endpoint instead of many

Limitations

- Harder to cache at the HTTP level
- More complex server (resolvers, N+1 problem)
- Deep/nested queries can be exploited if unguarded

WebSocket

REST/HTTP is request-driven — the client must repeatedly ask "any new data?" (polling). For real-time apps like WhatsApp, polling every second from millions of users would overload servers. **WebSocket enables full-duplex, persistent, bidirectional communication** — either side can send a message anytime.

Handshake

- 1 Client sends a normal HTTP request with special headers: `Connection: Upgrade`, `Upgrade: websocket`
- 2 Server responds: `HTTP/1.1 101 Switching Protocols`
- 3 Connection upgraded — the same TCP connection stays open for continuous two-way messaging

Ready States

State	Value	Meaning
CONNECTING	0	Handshake in progress
OPEN	1	Connection live — can send/receive
CLOSING	2	Connection is shutting down
CLOSED	3	Connection fully terminated

Sending on a CLOSING or CLOSED socket throws an error or is silently dropped — it causes an application-level bug, not a server crash. Always check `readyState === 1` before sending.

Client-Side Code Example

```
const socket = new WebSocket("wss://chat.example.com/room/42");

socket.onopen = () => console.log("Connected");
socket.onmessage = (event) => console.log("New message:", event.data);

// Only send when connection is actually open
if (socket.readyState === WebSocket.OPEN) {
  socket.send(JSON.stringify({ text: "Hey there!" }));
}
```

Real-World Usage

Chat apps (WhatsApp Web, Slack), live order tracking, stock tickers, multiplayer games, collaborative tools (Google Docs-style live cursors).

Limitations

- Stateful connections are harder to horizontally scale — need sticky sessions or a pub/sub layer like Redis
- Some corporate proxies/firewalls don't handle the Upgrade handshake well

Webhooks

The inverse of REST. In REST, the client polls the server ("did the payment happen yet?"). In webhooks, the **server calls the client** the moment an event happens — you register a URL in advance, and the provider (e.g., Razorpay) POSTs to it when the event occurs.

Flow (Razorpay Payment Example)

- 1 You register `https://yourapp.com/webhook/payment` with Razorpay
- 2 Customer pays
- 3 Razorpay automatically sends a POST request to your URL with payment details
- 4 No repeated polling required

Security — Critical Concept

Since the webhook URL is public, anyone could send a fake POST pretending to be Razorpay. To prevent this, the provider signs the payload with a secret key, producing a header signature that your server recomputes and compares.

This signature is technically an **HMAC (Hash-based Message Authentication Code)** — a hash combined with a shared secret key, which is what makes it verifiable and tamper-proof.

Example Payload Razorpay Would POST to Your URL

```
POST /webhook/payment HTTP/1.1
Content-Type: application/json
X-Razorpay-Signature: 4c9f2b6e1a7d...

{
  "event": "payment.captured",
  "payload": {
    "payment_id": "pay_KjX92mQ",
    "amount": 34900,
    "currency": "INR",
    "status": "captured"
  }
}
```

Your handler recomputes the HMAC of the raw body using the shared webhook secret and compares it against `X-Razorpay-Signature` — if they don't match, the request is rejected.

Best Practices

- Always verify the signature before processing
- Respond quickly (2xx) to acknowledge, process asynchronously
- Make webhook handlers idempotent — events can be delivered more than once

Other Common Webhook Use Cases

- GitHub → CI/CD pipeline trigger on `push` or `pull_request`
- Stripe → subscription renewal / failed-payment notifications
- Shopify → new order created, inventory updated

gRPC

Full form: gRPC Remote Procedure Call | **Origin:** Built internally at Google (evolving from an older internal system called "Stubby"), open-sourced in 2015.

In microservice architectures (order service, payment service, notification service all calling each other), passing JSON text as in REST is comparatively slow and heavy at scale. gRPC uses **Protocol Buffers (Protobuf)** — a binary serialization format — instead of JSON.

gRPC/Protobuf is generally faster and more bandwidth-efficient than REST/JSON due to binary encoding and HTTP/2 multiplexing — published benchmarks vary roughly 2×–10× depending on payload and use case.

Protobuf Schema Example (.proto file)

```
service OrderService {
  rpc GetOrder (OrderRequest) returns (OrderResponse);
  rpc TrackOrder (OrderRequest) returns (stream LocationUpdate); // server streaming
}

message OrderRequest {
  string order_id = 1;
}

message OrderResponse {
  string order_id = 1;
  string status = 2;
  double total = 3;
}
```

This schema is compiled into client/server code in multiple languages (Python, Go, Java, Node) — both sides share a strict, generated contract instead of hand-writing JSON parsing.

Four Call Types

Type	Pattern	Example Use Case
Unary	1 request → 1 response	Simple API call
Server streaming	1 request → many responses	Live order tracking
Client streaming	many requests → 1 response	Chunked file upload
Bidirectional streaming	many ↔ many	Real-time chat

Advantages

- High performance, low latency — ideal for internal microservice calls
- Strong typing via Protobuf catches contract mismatches at compile time
- Native streaming support (REST needs WebSocket/SSE bolted on)

Limitations

- Not natively browser-friendly (needs gRPC-Web + proxy)
- Binary format is harder to debug without tooling

SOAP API

Full form: Simple Object Access Protocol | **Origin:** Late 1990s (~1998).

Uses **XML** exclusively for message format, with a strict structure: **Envelope** → **Header** → **Body**.

```
<soap:Envelope>
  <soap:Header> <!-- auth, routing --> </soap:Header>
  <soap:Body> <!-- request/data --> </soap:Body>
</soap:Envelope>
```

Why It's Considered Highly Secure

WS-Security provides three things together: Authentication (who you are), Digital Signatures (message wasn't tampered with), and Encryption (confidentiality). This is why **banks and enterprise/ERP systems** still use SOAP today.

RELEVANT TO YOUR WORK

This directly ties to your SAGE X3 SOAP integration work on the IPN GL454 pipeline (YEXPMOD, YSENDSFTP, YGETSFTP, YIMPMOD) — SOAP's security model is exactly why ERP/finance systems favor it over REST.

SOAP vs REST

Aspect	SOAP	REST
Format	XML only	JSON, XML, plain text
Protocol	HTTP, SMTP, others	HTTP only
Security	Built-in (WS-Security)	Relies on HTTPS + external auth
Performance	Heavier (verbose XML)	Lighter (compact JSON)
Use today	Banking, legacy ERP	Public web/mobile APIs

API Gateway & Production Infrastructure

A raw API endpoint is just a function running on a server. In production, you never expose that function directly to the internet — you put an **API Gateway** in front of it. Think of the gateway as a middleware layer that does essential work before a request ever reaches your application code.

What the Gateway Handles

Responsibility	What It Does
Rate limiting	Rejects excess requests before they overwhelm your servers (e.g., a client sending 10,000 requests at once)
Authentication	Validates API keys, JWTs, or other credentials before allowing the request through — including plan-based access (e.g., a Basic-tier user can't reach Pro-only endpoints)
Routing	Directs incoming requests to the correct backend service
Logging & metrics	Captures response time, error rates, and usage per client for monitoring

Monitoring & Documentation Tools

Purpose	Common Tools
Monitoring / observability	Datadog, Prometheus, Grafana
API documentation	Swagger / OpenAPI

WHY THIS MATTERS

Monitoring and documentation aren't optional extras — monitoring tells you the instant something breaks in production, and documentation is what lets other developers (or your future self) actually use the API correctly.

API Frameworks

Whatever the language, building an API generally follows the same pattern: **define endpoints, accept requests, validate inputs, run logic, return responses**. The framework just changes how much of that is automated for you.

Framework	Language / Ecosystem	Known For
Flask	Python	Lightweight, beginner-friendly, minimal boilerplate
FastAPI	Python	High performance, automatic OpenAPI/Swagger docs, async support
Django REST Framework	Python (Django)	Full-featured, great for large Django applications
Express.js	Node.js	Minimal, unopinionated, the Node.js standard
Spring Boot	Java	Enterprise-grade, widely used in large Java systems
ASP.NET	C# / .NET	Standard choice in Microsoft-centric environments

Minimal Flask Example

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})

@app.route("/summarize", methods=["POST"])
def summarize():
    text = request.json.get("text", "")
    return jsonify({"summary": text[:100]})
```

Testing Tools

Postman is the standard tool for testing APIs without writing any frontend code — choose the method, enter the URL, add headers, attach a body if needed, and inspect the response directly.

Additional Expert Insights

AuthN vs AuthZ — Authentication (who you are) and Authorization (what you're allowed to do) are distinct concerns, often conflated.

Idempotency — A recurring theme across REST/gRPC/Webhooks; same request repeated = same effect. Strong interview signal.

API Gateway pattern — Production systems rarely use one API type alone; a gateway often routes REST/GraphQL externally while services talk gRPC internally.

Server-Sent Events (SSE) — A lighter one-way alternative to WebSocket for server → client streaming (e.g., notifications).

Rate limiting & throttling — Applies across all API types; a standard production consideration.

Bonus — MCP (Model Context Protocol) — a newer, separate standard (created by Anthropic, Nov 2024) for how AI models call external tools/data. It's not a peer to REST/SOAP/gRPC — it's an application-layer protocol built on top of them — but worth knowing given how fast AI tool-calling is spreading.

Frequently Confused Concepts

Confusion	Clarification
REST vs RESTful	REST = architectural style; "RESTful API" = one that follows REST principles
PUT vs PATCH	PUT replaces the whole resource; PATCH updates part of it
WebSocket vs Webhook	WebSocket = persistent two-way connection; Webhook = one-time POST triggered by an event
gRPC vs REST	gRPC = binary + HTTP/2 for internal speed; REST = text + broad compatibility
Hash vs HMAC	A hash can be recomputed by anyone; an HMAC needs a shared secret, making it tamper-proof

Interview Questions

Q1. What are the four main HTTP methods, and what does idempotency mean? Why is PATCH different from PUT?

A: GET (read), POST (create), PUT (full replace), DELETE (remove) — plus PATCH (partial update). An operation is **idempotent** if calling it once or many times with the same input produces the same end state (GET, PUT, DELETE are idempotent; POST and PATCH generally are not). PUT expects the *entire* resource representation and overwrites it; PATCH sends only the fields that changed, leaving the rest untouched — e.g., updating just a user's email via PATCH won't wipe their other profile fields the way a naive PUT could.

Q2. Why would you choose GraphQL over REST for a mobile app? What's the N+1 query problem?

A: Mobile clients are bandwidth- and battery-constrained, so avoiding over-fetching matters — GraphQL lets the client request exactly the fields a given screen needs in a single round trip, instead of hitting several REST endpoints. The **N+1 problem** is a server-side pitfall: resolving a list of N items (e.g., 20 orders) by then making one extra query per item to fetch related data (e.g., each order's restaurant) results in 1 + N database calls instead of a single batched one. It's typically solved with batching/caching tools like Facebook's **DataLoader**.

Q3. Walk through the WebSocket handshake. What are the four connection states?

A: The client sends a normal HTTP request with `Connection: Upgrade` and `Upgrade: websocket` headers plus a `Sec-WebSocket-Key`. If the server supports it, it replies `HTTP/1.1 101 Switching Protocols` and the same TCP connection is reused for a persistent, full-duplex channel — no new connection is opened. States: **CONNECTING (0)** handshake in progress, **OPEN (1)** ready to send/receive, **CLOSING (2)** shutdown initiated, **CLOSED (3)** connection ended.

Q4. How would you secure a webhook endpoint against spoofed requests?

A: Verify the provider's signature (HMAC) on every incoming request before trusting the payload — recompute the HMAC of the raw request body using a shared secret and compare it to the header the provider sent (e.g., `X-Razorpay-Signature`), using a constant-time comparison to avoid timing attacks. Additionally: only accept HTTPS, validate the payload schema, and make handling idempotent using an event/transaction ID so re-delivered events aren't double-processed.

Q5. Why does gRPC use HTTP/2 and Protocol Buffers? What are its four call types?

A: HTTP/2 provides **multiplexing** (many concurrent streams over one TCP connection, avoiding head-of-line blocking) and header compression — both reduce overhead for high-frequency microservice-to-microservice calls. Protobuf is a compact binary format with a strict, versioned schema — smaller and faster to (de)serialize than text JSON, and it catches contract mismatches at compile time. Call types: **Unary** (1 → 1), **Server streaming** (1 → many), **Client streaming** (many → 1), **Bidirectional streaming** (many ↔ many).

Q6. Why do banks still use SOAP instead of REST? What does WS-Security provide?

A: SOAP has a formal, strictly-typed contract (WSDL) and a mature, standardized security layer built into the message itself rather than relying purely on transport-level HTTPS. **WS-Security** bundles **authentication** (identity), **digital**

signatures (tamper-evidence/non-repudiation), and **encryption** (confidentiality) directly into the XML envelope — valuable for legacy enterprise systems (banking, insurance, ERP like SAGE X3) where the message may pass through multiple intermediaries, not just a single direct HTTPS hop.

Q7. Design question: Building a food delivery app — which API type for (a) menu browsing, (b) live order tracking, (c) payment confirmation, (d) internal service-to-service calls?

A: (a) **REST or GraphQL** — standard read-heavy, cacheable data. (b) **WebSocket** — needs continuous, low-latency, two-way updates as the delivery partner moves. (c) **Webhook** — the payment gateway should push a confirmation the instant it happens rather than your app polling. (d) **gRPC** — fast, strongly-typed calls between internal services like order, payment, and notification.

Q8. Why is JWT-based authentication described as "stateless," and what's the trade-off?

A: A JWT is self-contained and cryptographically signed, so the server can verify identity and permissions just by checking the signature — no database lookup needed on every request. This makes it fast and easy to scale horizontally. The trade-off: since the server isn't tracking sessions, a stolen token remains valid until it expires, which is why short-lived tokens plus a refresh-token flow are standard practice.

Quick Revision & Key Takeaways

REST

URL + Method + Headers + Body. Stateless. GET/POST/PUT/PATCH/DELETE.

GraphQL

Single endpoint, client defines response shape. Query / Mutation / Subscription.

WebSocket

Persistent full-duplex connection after HTTP "upgrade" handshake.

Webhooks

Inverse of REST — server pushes to a registered URL, secured via HMAC.

gRPC

Binary (Protobuf) + HTTP/2 multiplexing. 4 call types. Built for microservices.

SOAP

XML, strict envelope, WS-Security (AuthN + Signature + Encryption). Banking/ERP.

API Gateway

Rate limiting, auth, routing, logging — sits in front of every production API.

Frameworks

Flask/FastAPI/Django (Python), Express (Node), Spring Boot (Java), ASP.NET (.NET).

Key Takeaways

1. Every API type solves a specific communication problem — there's no single "best" type, only the best fit.
2. REST remains the default for public-facing APIs due to simplicity and tooling.
3. GraphQL solves over/under-fetching at the cost of caching simplicity.
4. WebSocket and Webhooks both remove polling, but solve opposite problems.
5. gRPC dominates internal microservice communication for performance.
6. SOAP persists in regulated domains — directly relevant to your SAGE X3 SOAP integration work.
7. Production APIs are rarely "raw" — a gateway, monitoring, and documentation layer sits around all of the above regardless of which type you pick.